Hackathon Technical Solution Report

Field	Details
Problem ID	PS-01
Problem Title	Robust Low-Light Image Restoration and Enhancement
Team Name	Pixel Pirates
Team Members	Shreeya Koppa Santosh-1RUA24CSE0439 Sinchana N-1RUA24CSE0460 Sharat Havanur-1RUA24CSE0424 Srusti KS-1RUA24CSE0479
GitHub Repository	https://github.com/shreeyaks02-png/LowLight_Image_Enhancement
Live Demo / Deployment	CV_video_recording.mp4 - Done in Streamlit

---

1. Problem Understanding & Solution Strategy

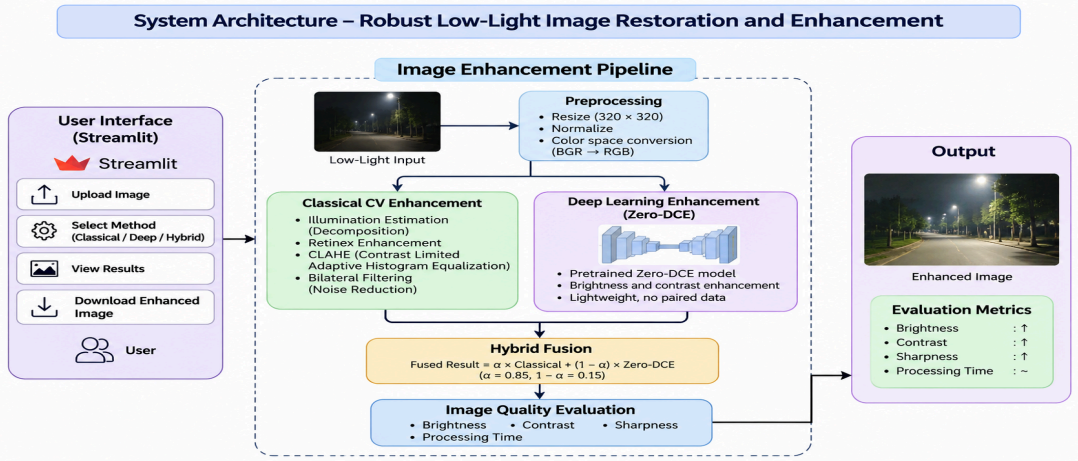
- **Core Technical Challenge:** Low-light images often have low brightness levels, low visibility, low contrast, increased noise levels, colour distortion and lack detail. Simply increasing brightness can create too much noise in dark parts and create washed out bright sections. The aim of this project is to increase visibility and contrast whilst maintaining good edges and somewhat natural colours. The application currently does inference on the CPU so processing time is a constraint.

- **Proposed Methodology:** The system accepts input RGB images that undergo image enhancement using both classical computer vision techniques and the pre-trained Zero-DCE Deep Learning model. The classical image processing techniques include adaptive gamma correction, Retinex-based enhancement, CLAHE, and bilateral filtering. Weighted fusion is then applied to merge the two images which are then

slightly sharpened to form the hybrid enhanced image. This facilitates comparison of the enhancements from each technique alongside that of the hybrid image.

2. Technical Architecture & Implementation

The code has been structured into 4 files: app.py for the streamlit app, src/classical.py for the classical enhancement, src/deep_learning.py for Zero-DCE inference and src/quality_metrics.py for the image metrics. The model has been saved at models/Epoch99.pth.



2.1 Classical Algorithmic Baseline

- **Algorithms Utilized:**

  1) Gaussian Blur to get an estimation of lighting using the greyscale channels.

  2) Adaptive Gamma Correction.

  3) In Retinex fashion we apply a filter to the LAB Lightness channel.

  4) Apply CLAHE to improve local contrast of the LAB Lightness channel.

  5) Bilateral Filter to reduce noise.

- **Adaptive Selection Logic:** Adaptive Selection: Calculate the mean brightness from the greyscale channel. Set gamma to clip(1-mean_brightness, 0.4, 0.9). Mean_brightness has been normalised to between 0 and 1. Gaussian sigma, CLAHE clip limit and tile size, and bilateral filter parameters are hard coded in this version.

2.2 Deep Learning Pipeline

- **Model Architecture:** The model is a convolutional neural network, called Zero-DCE (Zero-Reference Deep Curve Estimation). It is trained on models/Epoch99.pth to make

per-pixel curve estimations. Inference is carried out on the CPU. Large input images are scaled down to have the largest side at a certain length. The enhanced image is then scaled back to the original size. Colour preservation is also applied.Why this model: Zero-DCE was specifically designed for low light enhancement. It was originally trained without requiring paired low light and normal lighting images and therefore provides a learning based enhancement pathway to compare to the classical one.

- **Loss Functions & Training:** As we are using a pre-trained checkpoint in this project, no training will be done on the network in the streamlit app. The original Zero-DCE method uses zero-reference losses often discussed as Spatial Consistency Losses, Exposure Control Losses, Color Constancy Losses and Illumination Smoothness Losses. As this wasn't specified in any of the implementation details, we cannot say that this checkpoint has been trained on our dataset, with the specified configuration and losses.

2.3 Hybrid integration and adaptive mechanism

- **Fusion Strategy:** The classical and Zero-DCE outputs are blended together with a pixel-wise weight in app.py. The most recent version of app.py uses 85% classical and 15% Zero-DCE, followed by a Gaussian sharpening. Screenshots from earlier tests have been taken using the previous weight so the values below are based on this test and may not be accurate from using the new 85/15 weight.

- **Decision Logic:** The current system is using a set weight between the two. The classical helps to keep local edges and reduce some noise by filtering. The Zero-DCE provides learned lighting and contrast changes. However, there will be more edge variation due to the sharpening and potentially more noise.

---

3. Experimental Benchmarks & Evaluation

3.1 Quantitative Benchmarks

The following values are from one recorded Streamlit test image. They are descriptive no-reference measurements computed by src/quality_metrics.py, not PSNR, SSIM, LPIPS or NIQE.

Approach / Model Variant	Latency (ms / frame)
Classical Baseline	`660.9 ms`
Deep Learning Only	`24,619.6 ms`
Proposed Hybrid System	`25,296.2 ms`

The metrics of PSNR, SSIM, LPIPS and NIQE have not been included. PSNR and SSIM require the appropriate reference image to be used. Brightness is based on the average grey intensity, contrast is calculated from the standard deviation of the greyscale and sharpness is determined by the variance in the laplacian of the greyscale. Sharpness will not necessarily indicate that the image has better perceptual quality as it can be increased by noise and strong edges.

- **Analysis:** In this case study, Classical CV increased brightness and sharpness from 18.72 and 600.21 to 61.22 and 778.13 respectively. Zero-DCE had the highest contrast at 48.29 and lowest Laplacian variance of sharpness at 12.84. The hybrid result had improved brightness (60.68) and contrast (40.51) but reduced sharpness (723.21). The hybrid approach took much longer than the classical method, as there is inference taking place via deep learning. This is specific to the chosen image and cannot be generalised from one test.

Image Quality Evaluation Results

Method	Brightness	Contrast	Sharpness (Laplacian variance)
Original Image	18.72	31.83	600.21
Classical CV	61.22	39.60	778.13
Zero-DCE	57.38	48.29	12.84

Hybrid Enhancement	60.68	40.51	723.21

3.2 Visual Outputs & Intermediate Representations

This system provides visual feedback demonstrating the effectiveness of different enhancement techniques. The original image uploaded is shown alongside enhanced images from the Classical Computer Vision pipeline, pre-trained Zero-DCE model and proposed Hybrid System. The output from the Classical CV can be seen to enhance brightness, contrast and sharpness using adaptive gamma correction, retinex based enhancement, CLAHE and bilateral filtering. The output from the Zero-DCE can be seen to enhance the image using a Deep Learning based approach. The output from the Hybrid method combines the two enhancement techniques together using weighted fusion (85% Classical CV & 15% Zero-DCE). It then applies some mild sharpening to improve the clarity. Brightness, contrast, sharpness and processing time are also shown in the application to allow comparison between the enhancement techniques. This provides visual feedback to allow assessment of how effective each enhancement technique is. Particularly for very dark images and uneven lighting conditions. The lighting estimation is calculated internally within the classical pipeline so does not show in the application.

1. **Extreme low-light:**

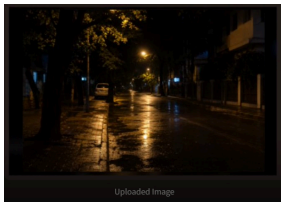


Figure 3.2(a) – Original Low-Light Input: The uploaded dark street image.

2. **Non-uniform / backlit scenes:**

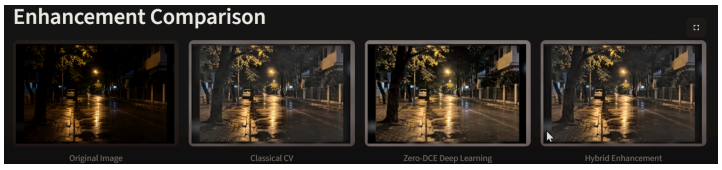


Figure 3.2(b) – Enhancement Comparison: The screenshot showing Original Image, Classical CV, Zero-DCE and Hybrid Enhancement side by side.

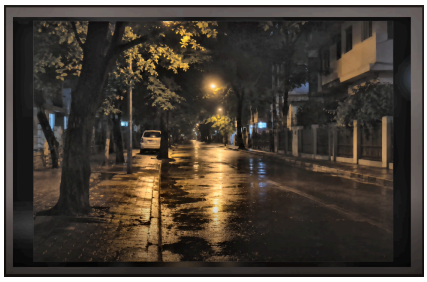


Figure 3.2(c) – Proposed Hybrid Output: The enlarged final hybrid-enhanced image.

3.3 Ablation Study

- There was no formal ablation study which disabled one module at a time, but we have included here a four way comparison between the original image, classical only result, Zero-DCE only result and hybrid result. This is not an ablation study, it is a comparison of the different methods. You would need to re-run the test images using each of the chosen components disabled to conduct a proper ablation study. All other settings would remain unchanged.

---

4. Failure Modes & Limitations

- **Observed Failure Case:** Limitations have been observed throughout the tests. The Zero-DCE algorithm has a low level of Laplacian variance sharpness when compared to the others. The algorithm could magnify noise in very dark areas when a classical brightness and contrast enhancement is applied. The Zero-DCE algorithm takes longer to infer on the CPU. The constant fusion ratio may not be optimal on images with varying exposure or lighting.
- **Root Cause & Mitigation:** Increasing the intensity can cause sensor noise to be more visible. Bilateral filtering and other smoothing techniques can reduce small textures within the image. Laplacian variance can be affected by useful edges and noise. The Zero-DCE model performs slower on the CPU. Improvements can be made to the amount of noise removal. Additional fusion weights based on the image can be added to increase performance. Colour preservation can be verified using varied scenes. Future improvements can be made to optimise the model.

---

5. Deployment & Quickstart Guide

Quickstart Commands

```shell
#1. Clone the repository
git clone <your-github-repository-url>

# 2. Navigate to the project directory
cd LowLight_Image_Enhancement

# 3. Create a virtual environment
python -m venv venv

# 4. Activate the virtual environment
venv\Scripts\activate

# 5. Install dependencies
pip install -r requirements.txt

# 6. Run the Streamlit application
streamlit run app.py
```

Deployment Format

This project is a local web application using Streamlit. Dependencies include Python, OpenCV, NumPy, PyTorch, Pillow, pandas, Altair. An image file (.JPG, .JPEG, .PNG) can be uploaded and enhanced. The four image outputs and quality comparisons will be displayed. Enhanced images can then be downloaded. This is currently being run on CPU.

The run_inference.py example given in the template cannot be used as the entry point of this repository is app.py. Unless an inference script is made available via CLI, the streamlit command should be used.

Student Submission Checklist

- ☑ Working GitHub repository link included with an updated `README.md`. - GitHub Repository: https://github.com/shreeyaks02-png/LowLight_Image_Enhancement
- ☑ No raw code pasted into this report.
- ☑ All table metrics match test runs reproducible via the repository.
- ☑ All team member details are accurately recorded.